Sebastian Banuelos- Ae103 Final Project

SkyGuard Technologies

Flight Control Preliminary Design Review

1 Introduction

At SkyMotive we are committed to developing cutting edge technologies for aerial transport applications. From urban air mobility to burrito delivery drones, we provide efficient and reliable solutions for the rapidly evolving drone industry. Our new line of racing drones features an advanced autonomous navigation system designed to push the limits of speed and precision.

SkyMotive's newest line of racing drones features a novel autonomous navigation system for optimal trajectory tracking. By implementing an optimization algorithm, this system is able to generate cost-efficient trajectories given desired waypoints provided by the user. To ensure smooth and stable flight, these trajectories are specifically designed to minimize snap — the second derivative of acceleration with respect to time. Minimizing snap enhances motion smoothness and prevents abrupt changes in force that could compromise the drone's stability.

On the hardware side, our drones are quipped with GPS and IMU modules, which provide information about the drone's position, orientation, and velocity. Ultimately, these components enable full-state feedback for our custom attitude controller, designed to track the planned waypoints from the trajectory generation. Furthermore, this controller is capable of rejecting unmodeled disturbances, such as wind gusts.

The following document presents the technical foundation of our drone controller, including mathematical modeling, control design, and simulation results demonstrating its efficacy in real-world scenarios.

2 Trajectory Generation

In this section, we outline the mathematical framework used to generate optimal trajectories for SkyMotive's racing drones. The trajectory generation process aims to find a smooth path between desired waypoints while minimizing snap, which enhances stability and performance.

Problem Formulation
The trajectory generation problem is formulated as a quadratic programming (QP) problem that minimizes the snap along the desired path. Snap, which is the fourth derivative of position with respect to time, is minimized to ensure smooth transitions and reduce abrupt force changes during flight.

Given the desired waypoints $\mathbf{p}_0, \mathbf{p}_1, \ldots, \mathbf{p}_N$ with associated times $t_0, t_1, \ldots, t_N$, we aim to find a piecewise polynomial trajectory for each dimension $x(t), y(t), z(t)$. For each segment i between waypoints $\mathbf{p}_i$ and $\mathbf{p}_{i+1}$, the trajectory is represented by a polynomial of order n:

$$\mathbf{p}_i(t) = a_0 + a_1(t - t_i) + a_2(t - t_i)^2 + \ldots + a_n(t - t_i)^n$$

The objective is to minimize the integral of the square of the snap over the entire trajectory:

$$J = \int_{t_0}^{t_N} \left(\frac{d^4\mathbf{p}(t)}{dt^4} \right)^2 dt$$

Quadratic Programming Formulation
To formulate the QP problem, we need to discretize the trajectory generation problem. This involves defining a **cost function** and a set of equality constraints corresponding to initial, final,

and waypoint conditions.

The cost function is constructed by integrating the square of the snap for each segment. For a single segment, this becomes:

$$J_i = \int_{t_i}^{t_{i+1}} \left(\frac{d^4 \mathbf{p}_i(t)}{dt^4} \right)^2 dt$$

The total cost is then the sum of individual segment costs:

$$J = \sum_{i=0}^{N-1} J_i$$

The cost function can be rewritten as a quadratic form:

$$J = \mathbf{a}^T Q \mathbf{a}$$

where $\mathbf{a}$ is the vector of polynomial coefficients for all segments, and Q is a matrix that encodes the snap minimization terms for the entire trajectory.

Constraints

The trajectory generation problem also includes several constraints:

1. Waypoint Constraints: The trajectory must pass through specified waypoints.
2. Continuity Constraints: The trajectory must be continuous in position, velocity, acceleration, and jerk at the boundaries between segments.
3. Boundary Conditions: Initial and final states (position, velocity, acceleration, and jerk) are specified.

These constraints can be expressed as a set of linear equations:

$$A\mathbf{a} = \mathbf{b}$$

where A is a matrix encoding the constraints and $\mathbf{b}$ is a vector representing the desired waypoint values and derivatives.

In MATLAB, the Quadratic Programming problem presented above can be solved using quadprog(), given an appropriate definition of the objective function and system constraints.

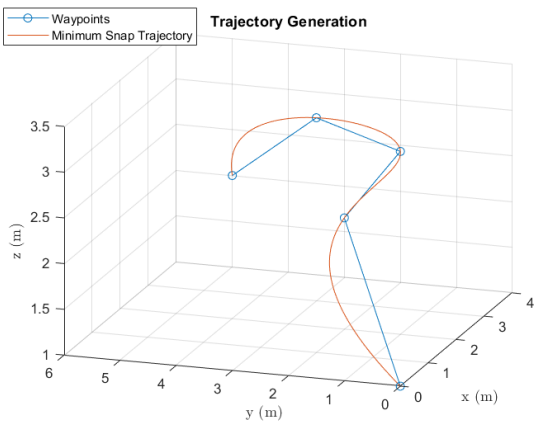

Figure 1: Estimated States

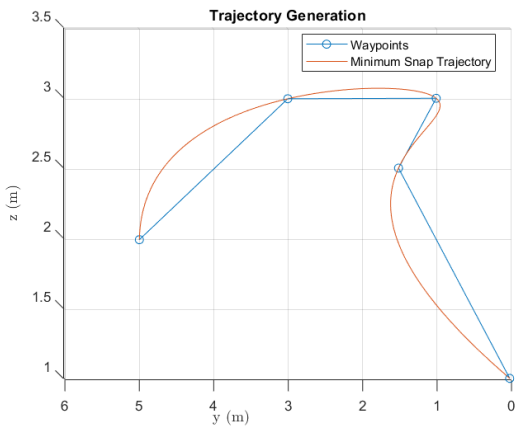

Figure 2: Actual States

Figure 1. and *Figure 2.* show the optimal trajectory solution given the desired waypoints

$$x_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad x_2 = \begin{bmatrix} 1 \\ 1.5 \\ 2.5 \end{bmatrix} \quad x_3 = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \quad x_4 = \begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix} \quad x_5 = \begin{bmatrix} 4 \\ 5 \\ 2 \end{bmatrix}$$

Note the aggressiveness of the trajectory can be modified by tuning the desired timepoints at each waypoint.

3 System Dynamics

In this section, we present the equations of motion governing the quadrotor system, including the relevant coordinate frames and their relationships. The quadrotor dynamics are derived using Newton-Euler formalism, where we consider both translational and rotational motion.

3.1 Coordinate Frames

The system dynamics are expressed in terms of two coordinate frames:

1. Inertial Frame ($\mathcal{I}$): A fixed, world coordinate system defined by axes (X, Y, Z).
2. Body Frame ($\mathcal{B}$): A moving coordinate system attached to the quadrotor, defined by axes (x, y, z). The thrust generated by the propellers acts along the z-axis of this frame.

3.2 Translational Dynamics

The translational motion of the quadrotor is governed by Newton's second law, which relates the external forces acting on the quadrotor to its linear acceleration in the inertial frame:

$$m\ddot{\mathbf{r}} = \mathbf{F}_{\text{gravity}} + \mathbf{F}_{\text{thrust}}$$

Breaking this down, the gravitational force is given by:

$$\mathbf{F}_{\text{gravity}} = -mg\hat{z}$$

The thrust force generated by the rotors is aligned with the body frame's z-axis and can be expressed in the inertial frame by applying the rotation matrix R:

$$\mathbf{F}_{\text{thrust}} = R \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix}$$

Combining these forces, the translational equation of motion becomes:

$$m\ddot{\mathbf{r}} = -mg\hat{z} + R \begin{bmatrix} 0 \\ 0 \\ T \end{bmatrix}$$

3.3 Rotational Dynamics

The rotational motion of the quadrotor is governed by Euler's rotation equations, which describe the relationship between the applied torques and the angular acceleration:

$$I\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (I\boldsymbol{\omega})$$

Where:

- I is the moment of inertia matrix.
- $\boldsymbol{\omega}$ is the angular velocity vector in the body frame.
- $\boldsymbol{\tau}$ is the external torque vector, consisting of control torques generated by the motors.

The control torques are derived from the differences in thrust generated by the individual rotors. These can be expressed as:

$$\boldsymbol{\tau} = \begin{bmatrix} \tau_x \\ \tau_y \\ \tau_z \end{bmatrix}$$

4 Controller Design

The controller for the quadrotor is responsible for stabilizing and controlling both the translational and rotational motion of the system. By regulating position, velocity, orientation, and angular velocity, the controller ensures that the quadrotor follows a desired trajectory with high precision.

4.1 Control Objectives

The control objective is to track desired position , velocity , orientation , and angular velocity . The equations of motion governing the system are divided into translational and rotational dynamics.

4.2 Translational Control

The translational dynamics are given by:

$$m\ddot{\mathbf{r}} = -mg\hat{z} + R\begin{bmatrix}0 & 0 & T\end{bmatrix} \tag{1}$$

To achieve position control, a feedback control law is designed using a PD controller:

$$\mathbf{F}_{\text{des}} = m(\ddot{\mathbf{r}}_d + K_p(\mathbf{r}_d - \mathbf{r}) + K_v(\dot{\mathbf{r}}_d - \dot{\mathbf{r}})) \tag{2}$$

The desired thrust magnitude is then:

$$T = \mathbf{F}_{\text{des}} \cdot R\hat{z} \tag{3}$$

The desired orientation is calculated from the desired thrust direction:

$$\hat{z}d = \frac{\mathbf{F}\text{des}}{|\mathbf{F}_{\text{des}}|} \tag{4}$$

4.3 Rotational Control

The rotational dynamics of the quadrotor are governed by:

$$I\dot{\boldsymbol{\omega}} = \boldsymbol{\tau} - \boldsymbol{\omega} \times (I\boldsymbol{\omega}) \tag{5}$$

The control objective is to track the desired orientation and angular velocity . The orientation error is represented using:

$$R_e = R_d^T R \tag{6}$$

The attitude error vector is computed using the operator:

$$\mathbf{e}_R = \frac{1}{2}\left(R_d^T R - R^T R_d\right)^\vee \tag{7}$$

The angular velocity error is defined as:

$$e_\omega = \boldsymbol{\omega} - \omega_d \tag{8}$$

4.4 Control Law

The control torques are generated using a PD controller for attitude regulation:

$$\boldsymbol{\tau} = -K_R e_R - K\omega e_\omega \tag{9}$$

Where:

- K_R is the proportional gain matrix for attitude error.
- K_ω is the derivative gain matrix for angular velocity error.

The following script implements the system dynamics and the controller presented above using a Forward Euler Scheme.

```
num_pts = 2000;
‘

% Define drone parameters
m = 1;
g = -9.81;
Ixx = 1e-6;
Iyy = 1e-6;
Izz = 1e-6;

u1 = zeros(1, num_pts);
u2 = zeros(1, num_pts);
u3 = zeros(1, num_pts);
```

```
u4 = zeros(1, num_pts);

t = linspace(1, dt*num_pts, num_pts);

initial = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0];
desired = [0, 10, 10, 0, 0, 0, 0];

x(1) = initial(1);
y(1) = initial(2);
z(1) = initial(3);

x_dot(1) = initial(4);
y_dot(1) = initial(5);
z_dot(1) = initial(6);

phi(1) = initial(7);
theta(1) = initial(8);
psi(1) = initial(9);

p(1) = initial(10);
q(1) = initial(11);
r(1) = initial(12);

% Define tuning parameters
Kp = 2 * diag([1, 1, 1]);
Kv = 4 * diag([1, 1, 1]);
Kr = 0.0001 * diag([1, 1, 1]);
Kw = 0.00001 * diag([1, 1, 1]);


for i = 1:num_pts - 1
    % Compute errors
    ep = desired(1:3)' - [x(i), y(i), z(i)]';
    ev = desired(4:6)' - [x_dot(i), y_dot(i), z_dot(i)]';

    % Desired force calculation
    F_des = Kp * ep + Kv * ev - [0; 0; m * g];

    % Desired z_B vector
    z_B_des = F_des / norm(F_des);

    % Compute desired orientation (Euler angles)
    phi_des = asin(-z_B_des(2));
    theta_des = atan2(z_B_des(1), z_B_des(3));
    psi_des = desired(7);  % Keep yaw fixed for now

    % Compute desired rotation matrix
    WR_B_des = angle2dcm(psi_des, theta_des, phi_des, 'ZYX');

    % Compute current rotation matrix
    WR_B = angle2dcm(psi(i), theta(i), phi(i), 'ZYX');

    % Compute rotation error matrix
    R_error = WR_B_des * WR_B';

    % Extract orientation error vector
    er = 0.5 * [R_error(3,2) - R_error(2,3);
                R_error(1,3) - R_error(3,1);
                R_error(2,1) - R_error(1,2)];
    ew = [p(i), q(i), r(i)]';

    % Compute control moments
    u = - Kr * er - Kw * ew;
    u2(i) = u(1);
    u3(i) = u(2);
    u4(i) = u(3);

    % Compute thrust input
    z_B = [sin(theta(i)), -cos(theta(i)) * sin(phi(i)), cos(theta(i)) * cos(phi(i))];
    u1(i) = dot(F_des, z_B');

    % Update Translational Dynamics
    x(i + 1) = x(i) + dt * x_dot(i);
    y(i + 1) = y(i) + dt * y_dot(i);
    z(i + 1) = z(i) + dt * z_dot(i);

```

```
    x_dot(i + 1) = x_dot(i) + dt * u1(i)/m * z_B(1);
    y_dot(i + 1) = y_dot(i) + dt * u1(i)/m * z_B(2);
    z_dot(i + 1) = z_dot(i) + dt * (g + u1(i)/m * z_B(3));

    % Update Rotational Dynamics
    phi(i + 1) = phi(i) + dt * p(i);
    theta(i + 1) = theta(i) + dt * q(i);
    psi(i + 1) = psi(i) + dt * r(i);

    p(i + 1) = p(i) + dt * (((Iyy - Izz) / Ixx) * q(i) * r(i) + u2(i) / Ixx);
    q(i + 1) = q(i) + dt * (((Izz - Ixx) / Iyy) * p(i) * r(i) + u3(i) / Iyy);
    r(i + 1) = r(i) + dt * (((Ixx - Iyy) / Izz) * p(i) * q(i) + u4(i) / Izz);

end
```

In order to test the functionality of this controller, the following demonstration maintain the drone in a hovering state.

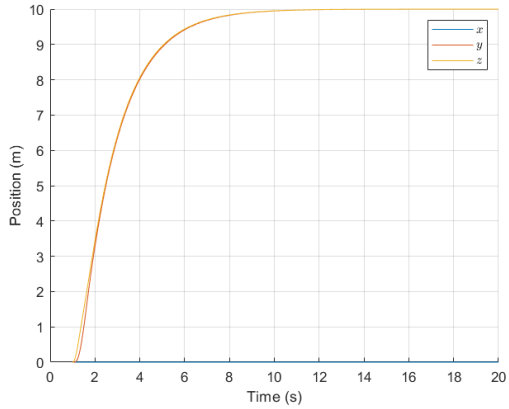

Figure 3: Position Response

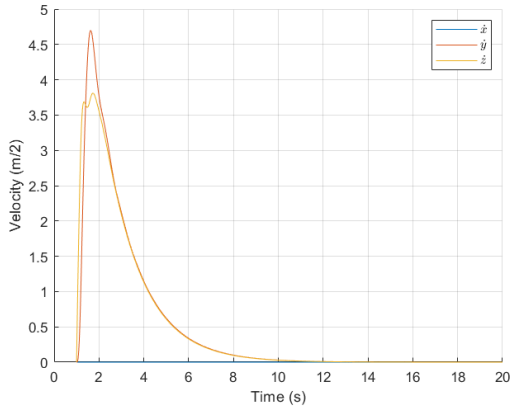

Figure 4: Velocity Response

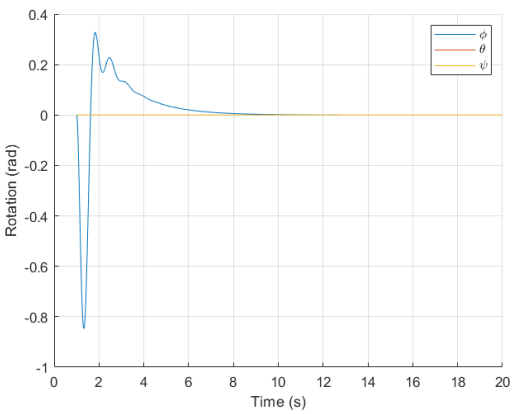

Figure 5: Position Response

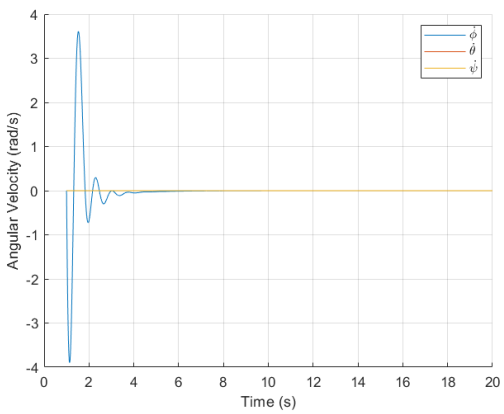

Figure 6: Velocity Response

5 Full System Integration

After implementing the trajectory optimization and system controller independently in MATLAB, the full-system was modeled in SIMULINK. This takes a set of desired waypoints as user input, which are then relayed to the Minimum Snap Trajectory Generator. This block computes the optimal trajectory solution and outputs the desired system state in real time.

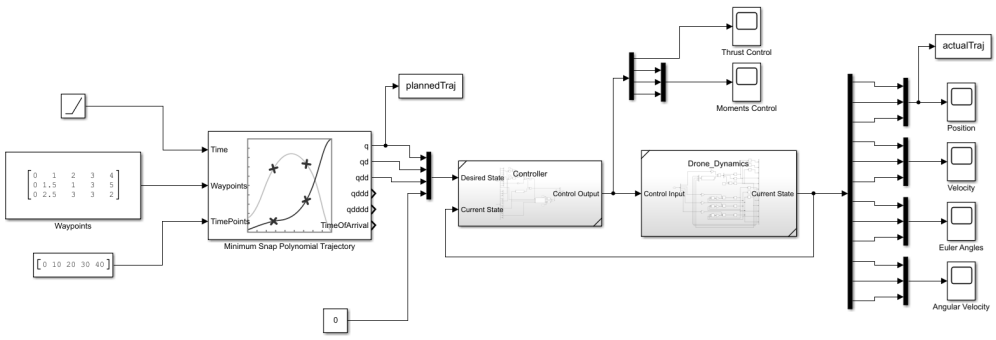

Figure 7: SIMULINK Model of Closed Loop System

A custom subsystem is used to compute the controller input from the current error states, given the current system state and the desired state in the trajectory.

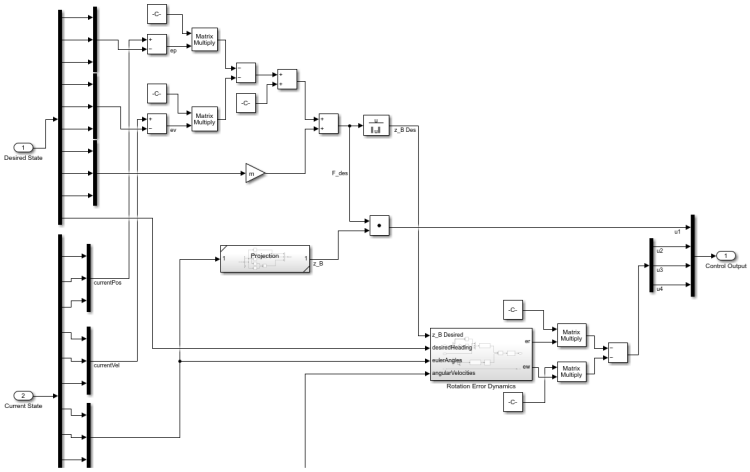

Figure 8: Controller Subsystem

Similarly, custom subsystem was used to model the dynamics of the drone, following the equations of motion presented earlier.

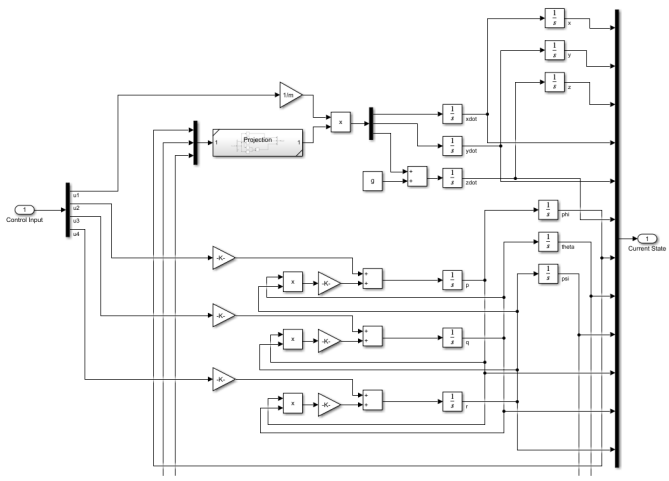

Figure 9: Drone Dynamics Subsystem

The performance of the system was tested using the trajectory presented in section (1) with the

following waypoints.

$$x_1 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad x_2 = \begin{bmatrix} 1 \\ 1.5 \\ 2.5 \end{bmatrix} \quad x_3 = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix} \quad x_4 = \begin{bmatrix} 3 \\ 3 \\ 3 \end{bmatrix} \quad x_5 = \begin{bmatrix} 4 \\ 5 \\ 2 \end{bmatrix}$$

$$t = \begin{bmatrix} 0 & 2 & 4 & 6 & 8 \end{bmatrix}$$

5.1 System Response

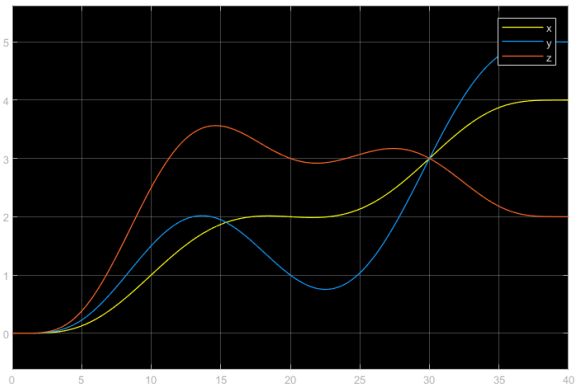

Figure 10: Position Response

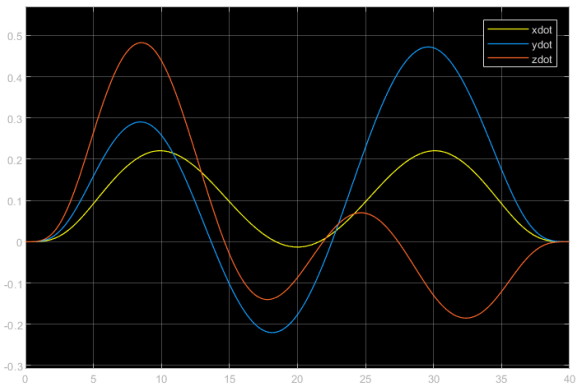

Figure 11: Velocity Response

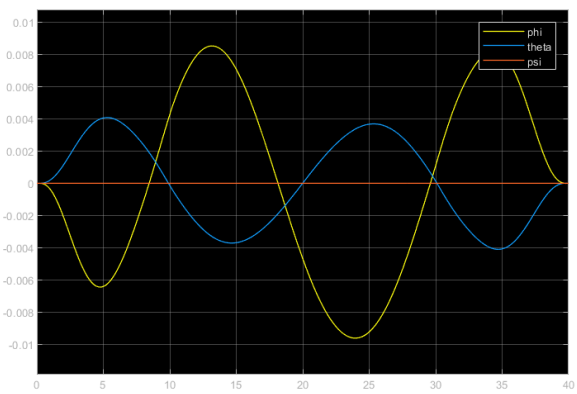

Figure 12: Orientation Response

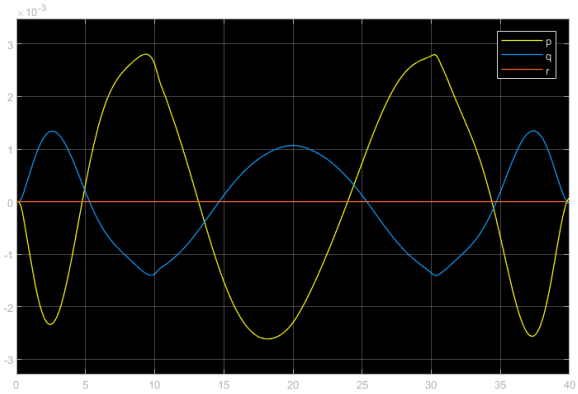

Figure 13: Angular Velocity Response

From the figures presented above, it can be seen the controller achieves a stable response driving the system to the desired waypoints at their give timepoint.

5.2 Tracking Performance

The controller was able to follow the pre-planned trajectory considerably well. During the following figures, it can be observed how the controller achieved a trajectory of ± 6 cm.

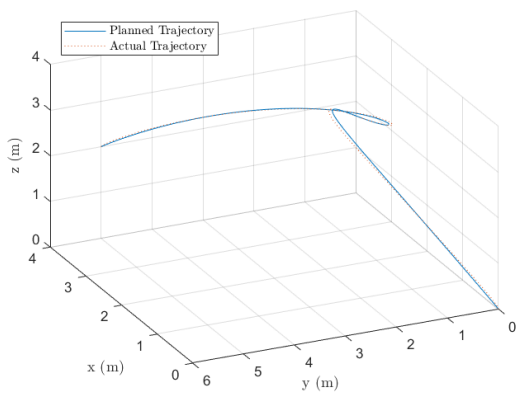

Figure 14: Trajectory Comparison

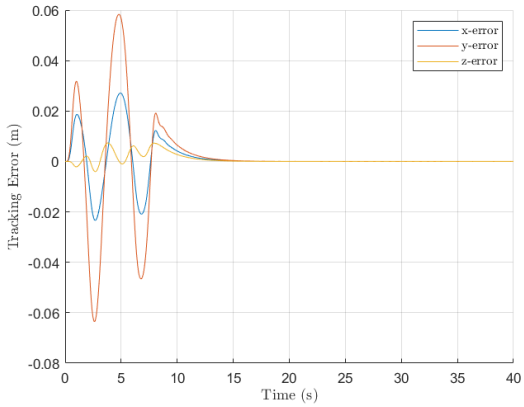

Figure 15: Error Response

It is important to note that the tracking performance of the system depends on aggressiveness of the trajectory. As we push the limits of the drone by reducing the time of arrival, the controller struggles to keep up with the planned trajectory.The simulation above had a time of arrival of 8 seconds.

For a time of arrival of 4 seconds, the tracking performance significantly decreases.

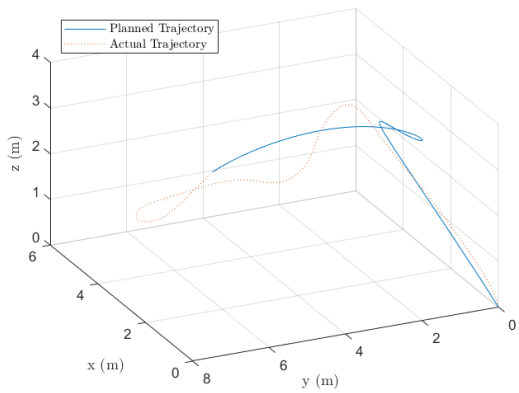

Figure 16: Trajectory Comparison

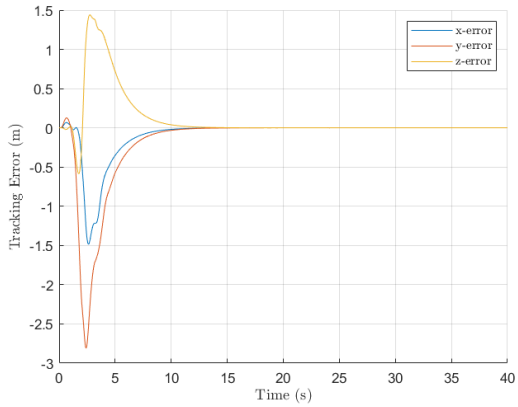

Figure 17: Error Response

For a time of arrival of 40 seconds, the tracking performance is almost flawless.

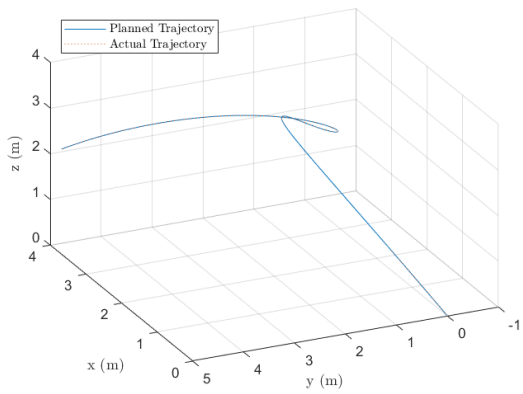

Figure 18: Trajectory Comparison

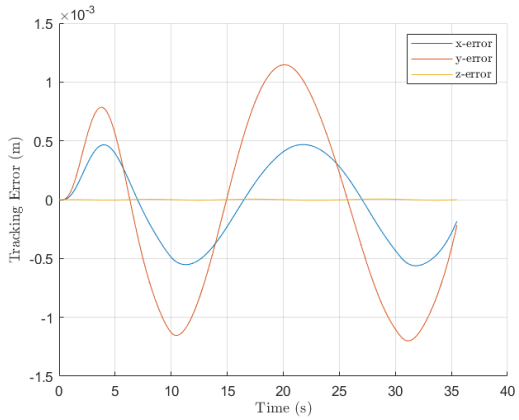

Figure 19: Error Response

6 Disturbance Rejection

Finally, we tested the performance of the controller under unmodeled disturbances. For this purpose, a periodic pulse generator was used to resemble gusts.

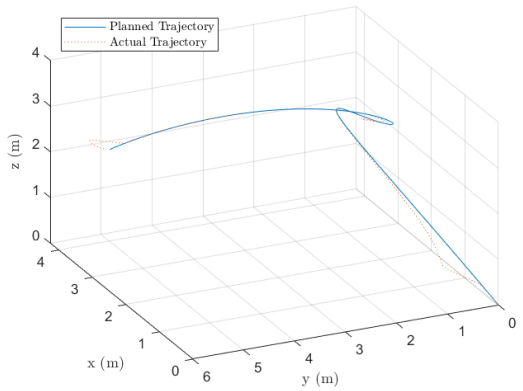

Figure 20: Estimated States

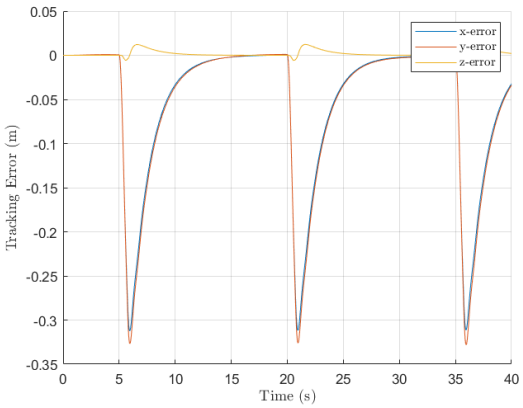

Figure 21: Actual States

From the plots above, it can bee seen how the wind gusts kick the drone off course, but the controller is able to compensate these disturbances and limit the tracking error.

Ultimately, the performance of the system could be improved by incorporating Model Predictive Control. When the drone deviates from its course, it attempts to realign with the trajectory that was generated at the beginning of the routine. However, this is no longer the most optimal trajectory, as the optimization constraints changed when the drone got off course. Therefore, in order to further improve the performance of this system, we are considering the possibility of periodically regenerating a new optimized trajectory across different waypoints.